2026 年华北五省（市、自治区）及港澳台大学生计算机应用大赛 移动互联网应用创新

移动互联网应用程序开发

【项目名称】 智座

（Android作品）

所在赛区:	河北赛区
所在学校:	沧州交通学院
团队名称:	苗苗
团队成员:	吴嘉森、刘彭宇、李冠桦
提交日期:	2026 年 11 月

参赛作品知识产权声明

参赛作品名称：智座（智能选座与导航一体化系统）（下称该作品）

本小组全体成员，就该作品声明如下：

1、该作品（含提交参赛的 App 应用、相关文档和介绍视频）是本小组成员的原创研究成果，不存在侵犯任何他人知识产权、涉及泄密或违反中华人民共和国法律法规的情形，且未在 2026 年 10 月 10 日前公开发布。

2、我们确认：该作品知识产权归本小组成员所有，且签署本声明的人员能够代表全体小组成员的意愿。

3、我们承诺：将仅由本小组成员参加大赛答辩。

4、大赛举办方为宣传大赛、推广参赛作品，以及为未来各届大赛的参赛选手提供参考等非盈利目的，可能需要以下列形式使用参赛作品，就此我们确认：

（1）☑ 同意 ☐ 不同意：将参赛作品的相关文档结集出版（含公开发行）；

（2）☑ 同意 ☐ 不同意：在相关网站（包括大赛官网及含优酷等第三方视频平台网站）上传和播放介绍视频或幻灯片；

（3）☑ 同意 ☐ 不同意：在大赛官方网站提供 APP 应用免费下载；

（4）作出上述许可，同样不违反本声明第 1 款的要求。

作品小组全体成员（签名）：

日期：2026 年 月 日

目　录

测试账号与访问方式 4
一、作品概述 4
二、作品可行性分析和目标群体 5
（1）可行性分析 5
（2）目标群体 7
三、作品功能与原型设计 7
（1）功能概述 7
（2）原型设计 8
（3）开发过程与各阶段设计 11
四、作品实现、难点及特色分析 12
（1）作品实现及难点 12
（2）特色分析 15
（3）核心代码与解析 17
五、团队介绍和人员分工 22
六、其他 23
（6）设备批量部署与测试规模 27
七、致谢 28

测试账号与访问方式

系统已预置下列测试账号，**无需注册即可直接登录体验全部功能**。评审期间请使用以下账号。

角色	用户名	密码	可体验的功能
超级管理员	**admin**	**SuperAdmin@123**	全部功能：后台管理、场所与平面图、传感设备绑定、学校与账号、AI 运营中心
普通用户	在登录页点「注册」，身份选「普通用户」，注册后直接登录		选座、预约、室内外导航、签到、AI 助手

访问方式

方式	地址 / 说明
网页端（推荐）	**https://zhinengzuo.site** 任意浏览器直接打开，无需安装
Android 应用	同一套页面运行在 Capacitor 壳内，加载同上地址；未安装时用网页端体验即可
大厅终端	https://zhinengzuo.site/terminal 常驻大屏页，无需登录

三步看到效果

① 用上表账号登录 → 进入「座位图」，看实时座位状态；
② 点任意空闲座位 → 选座 →「导航至此」，看室内路径；
③ 用管理员账号进「硬件 / 传感器调试」，可看到真实 ESP32 设备的上报与绑定。

> **说明：** AI 智能体与室外导航依赖第三方 Key（大模型、地图服务），若评审时未配置或额度用尽，页面会自动降级为规则文案，其余功能不受影响。

一、作品概述

「智座」是一套面向公共学习空间的**智能选座与导航一体化系统**，由「物联网感知层 + 云端决策层 + 移动应用层」三层构成：用低成本的嵌入式传感器实

时感知每一个座位是否有人，由云端完成座位调度、预约与鉴权，再通过 Android 移动应用把「哪里有空位」一路导航到「你坐下的那个位置」。

项目的起点来自团队在图书馆的真实体验。高校图书馆、自习室的座位供需矛盾长期存在：一方面，读者在馆内绕行数圈找不到空位，时间大量消耗在「找」上；另一方面，「占座不来」造成大量隐性浪费。而现有的解决方案存在两个共同缺口 ——

缺口一：只告诉你「有空位」，不告诉你「怎么走过去」。市面上的座位查询系统大多止步于一张状态图。而 GPS 在室内失效，读者拿到「3 楼 A 区有空位」这条信息后，仍然要在书架与隔断之间自行摸索，最后一米的体验是断的。

缺口二：室内定位方案成本过高。要做到室内定位，常规做法是部署蓝牙信标（iBeacon）或 UWB 基站，需要逐个安装、供电、维护与坐标标定，对学校而言是一笔持续投入，很难在普通教室、自习室规模化落地。

针对这两个缺口，本项目完成了三项关键工作：① 用 ESP32 + 超声波传感器做出单节点成本极低的占用感知终端，并实测出「3.3V 单电源直连、免分压电阻」的接法，把部署门槛进一步压低；② 提出并实现了**零信标室内定位方案** —— 以座位上已有的二维码作为定位锚点，配合手机自带的加速度计/陀螺仪做步态惯性推算，在不增加任何硬件成本的前提下提供连续位置；③ 把感知、预约、路径规划、移动端导航串成闭环，并引入大语言模型把原始传感数据转译为普通人能直接照做的建议。

系统已在真实环境中跑通全链路：ESP32 节点通过公网向云端上报占位状态，Android 应用可在任意网络下访问 **https://zhinengzuo.site** 完成注册、选座、预约、室内导航与室外导航，管理员可通过后台管理多校区场所、平面图、路网与传感设备。

二、作品可行性分析和目标群体

（1）可行性分析

技术可行性

系统所依赖的技术均为成熟且可验证的方案，且团队已完成全链路的真机实测，不存在「纸面可行、落地卡住」的风险。

感知层：ESP32（ESP32-D0WD 双核 240MHz，集成 2.4GHz Wi-Fi）搭配 HC-SR04 超声波测距模块，成本约 20 元/节点。团队实测确认：将模块 VCC 接 ESP32 的 3V3 引脚、TRIG 接 GPIO16、ECHO 接 GPIO27 **直连即可稳定出数，无需分压电阻**；50cm 以内的测距精度足以判断座位是否有人。节点

开机后自动连接 Wi-Fi、向云端注册并周期性拉取配置（绑定的座位号、判定阈值、上报间隔），全程免现场烧录。

云端层： Flask 3.1 + Flask-SQLAlchemy 3.1 + MySQL，通过 gunicorn（eventlet worker）承载，由 nginx 反向代理并绑定 HTTPS 域名，已稳定运行于云服务器，具备公网联通性（满足大赛第 5 条要求）。实时座位状态通过 WebSocket（Flask-SocketIO 5.5 + Socket.IO 4.7）推送到客户端，无需轮询。

移动层： 采用 Capacitor 7 将 Web 应用封装为原生 Android 应用，既保留一套代码同时服务浏览器与 App，又能调用扫码、语音识别、本地通知、地理定位等原生能力。室内导航采用「静态平面图 + 路网图 + A* 启发式寻路」，路径节点可吸附，支持跨楼层拼接；室外导航接入高德地图 JS API 2.0，并在未配置地图密钥时自动降级为自绘的「相对方位图」。

智能层： 接入 DeepSeek 大语言模型（模型 deepseek-chat，接口 api.deepseek.com/v1），用于把多源传感数据压缩成一句话结论、生成座位推荐理由。模型层做成可切换适配器，可在管理后台现场更换服务商与模型，也可切换为本地部署的模型服务。

操作可行性

用户侧的操作路径被压缩到极短：微信或浏览器打开链接即可使用，无需安装；安装 App 后，长按桌面图标可直接进入「找空座 / 扫码占座 / 我的预约 / 语音选座」四个快捷入口。日常最高频的两件事各只需一步 —— 想看空位，首页即显示实时状态；想坐下，扫桌上的二维码即可完成签到或占座。语音入口允许用户说一句话（如「找个空座」）由系统自动挑出座位并跳转导航，全程不需要手动检索。

管理侧同样按「可视化配置优先」设计：场所、楼层、平面图、路网、座位、传感设备、终端大屏的默认显示位置、AI 服务商与密钥，全部可在管理后台点击完成，不需要修改配置文件或重启服务。

经济可行性

本方案最突出的优势是**部署成本极低**。以一间 100 座的自习室为例：若采用蓝牙信标做室内定位，通常每 5–8 米需布设一个信标，100 座空间约需 15–20 个信标，加上电池更换与人工标定，一次性投入数千元且存在持续维护成本；而本方案**室内定位不依赖任何额外硬件**，只用座位上已有的二维码与手机自带传感器，这部分成本为零。

需要新增硬件的只有占用感知节点：单节点（ESP32 + HC-SR04 + 外壳供电）成本约 20–30 元，且由于采用 3.3V 直连免分压方案，接线与故障点更

少。软件全部基于开源技术栈，云端可运行在入门级云服务器上，年运营成本在数百元量级。

从收益侧看，系统能直接减少读者寻找座位的时间，并可通过预约与签到机制抑制「占座不来」，提高单位座位的实际使用效率，投入产出比清晰。

（2）目标群体

核心用户：在校学生（图书馆、自习室的主要使用者）。他们对空位信息的需求最强烈、使用频率最高。按使用场景可细分为三类：临时找座的学生，最关心「现在哪里有座、怎么最快走过去」，主要使用实时座位图、语音找座与室内导航；需要长时间固定学习的学生，更依赖预约与签到功能，希望提前锁定座位并在到馆后快速核验；对安静程度、靠窗与否有偏好的学生，则更看重按类型筛选座位与 AI 推荐理由。

次要用户：场馆管理人员。他们关注的是整体使用率、异常设备与座位纠纷处理，主要使用管理后台的场所与座位管理、传感设备监控、平面图与路网配置、AI 运营中心以及智能终端大屏。

延伸用户：校内其他公共空间。系统在数据模型上以「学校 → 场所 → 楼层 → 座位」分层，并支持多校隔离与学校管理员分级授权，同一套系统可直接复用到食堂、报告厅、研讨间、候车区等任何「座位资源紧张且需要引导」的公共空间。

三、作品功能与原型设计

（1）功能概述

功能模块	功能描述
实时座位图	以平面图为底图叠加座位，按空闲／已占用／已预约／传感器异常分色显示，通过 WebSocket 实时刷新，无需手动刷新页面。
座位预约与签到	用户选定座位与时长后提交预约，生成签到二维码；到馆扫码或点击签到完成核验，超时未签到自动释放，抑制「占座不来」。
室内导航	基于平面图与路网图做 A* 路径规划，支持手动选点、扫码定位、入口定位三种起点方式与座位／坐标两种终点方式，支持跨楼层路径拼接。
传感器导航（PDR）	以座位二维码为锚点，用手机加速度计做步态检测、陀螺仪／磁力计做朝向估计，实时给出步数、推算距离与朝向，为「最后一米」提供连续位置参考。

功能模块	功能描述
室外导航	接入高德地图绘制步行路线，展示目标建筑标记与直线距离；无地图密钥时自动切换为以用户位置为圆心的相对方位图，完全离线可用。
扫码占座	调用原生相机扫描座位二维码，直接完成占座或签到，免除手动检索与输入。
语音选座	调用系统语音识别获取一句话，本地解析意图（找座／预约／导航／取消／问答），「找空座」会自动挑选空闲座位并跳转导航。
实时提醒	通过本地通知推送预约到期提醒与座位状态变化，不清后台也能收到。
地理围栏	基于建筑经纬度判断用户是否进入／离开场馆范围，触发相应提示。
离线缓存	接口响应写入本机缓存，断网时仍可查看上一次的座位状态。
AI 智能体	全站可用的 AI 助手浮窗；把多源传感数据转译为「一句话结论」；为推荐座位生成可解释的理由；智能终端大屏定时生成场馆运行概述。
智能终端大屏	面向场馆入口的只读展示页，支持按管理员配置的默认场所与楼层打开，实时显示座位状态与 AI 概述，可切换楼层。
管理后台	场所与楼层管理、平面图与路网上传配置、座位批量管理与二维码生成、传感设备绑定与调试面板、学生批量导入、学校与权限管理、AI 运营中心、系统设置。
多校与权限	支持学生／普通用户／学校管理员／管理员四种身份注册，学校管理员只能管理本校资源，超级管理员全局管理。

（2）原型设计

运行平台：Android 移动终端（另提供浏览器版本） | 屏幕分辨率：适配 360×640 及以上 | 已测机型：iQOO Neo9S Pro（Android 16）



图 1 首页（AI 一句话结论 + 场所列表）



图 2 实时座位图（按状态分色 + 扫码占座）

图 1 首页界面：顶部显示当前应用名称与身份入口，主体为场馆横幅、「你在哪里」检索框与「全部场所」列表（含每个场所的座位数与空闲数），底部为五个主 Tab：首页、座位图、预约、导航、我的。右下角为 AI 助手浮窗，左下角为设置入口。应用外壳（顶栏、底部 Tab、侧边抽屉）只在 App 内生效，浏览器访问时保持原有网页版布局；导航项按角色动态生成，普通学生看不到「管理」入口。

图 2 实时座位图：以管理员上传的平面图为底图，按坐标叠加座位方块：绿色为空闲、红色为已占用、橙色为已预约、深灰为异常、浅灰带 ⊘ 为已关闭。顶部为楼层选择、搜索框与状态筛选，其后是状态统计（图示为 9 空闲 / 0 占用 / 0 锁定 / 0 异常 / 3 已关闭），右下角「扫码占座」按钮可直接调用相机扫座位二维码。已关闭的座位默认不出现在图面上，需要时把筛选切到「已关闭」单独查看。



图 3　室内导航（路径规划 + 传感器导航）



图 4　室外导航（定位与目标建筑）

图 3 室内导航：上半部分设置起点与终点，支持手动选点、扫码定位与入口定位；规划成功后显示路径节点序列并在地图预览上以蓝色折线绘制路径，红色圆点标注终点（图示为「演示大学 1 层」，共经过 8 个节点）。右下角为可折叠的「传感器导航」面板，显示步数、推算距离与朝向；扫码锚定后，按步态推算出的当前位置会以橙色圆点叠加在平面图上，并用虚线标出与锚点的位移关系。

图 4 室外导航：上方显示定位状态（图示精度约 31 米）与当前坐标；下方以高德地图为底图展示「我的位置」与目标建筑及直线距离。地图数据来源为高德地图，页面底部自带版权与审图号信息。未配置地图密钥、或步行路线服务不可用时，系统自动退化为自绘的**相对方位图** —— 以用户为圆心、各目标建筑按真实方位角与真实距离摆放，外圈标注距离刻度，完全离线可用。

管理后台：场所管理页可新增场所、设置地理位置（经纬度）、查看与编辑场所信息；平面图与路网配置页用于上传平面图、添加座位、绘制或由 CAD 图自动生成路网，是室内导航的数据来源（见后文图 5）。

（3）开发过程与各阶段设计

项目按软件工程的常规周期推进，共经历六个阶段。每个阶段的产出都直接决定了下一阶段的做法， 下面是每个阶段做了什么、遇到什么、怎么调整的。

① 需求分析。 起点是团队在图书馆的真实使用体验：读者绕行数圈找不到空位，「占座不来」造成隐性浪费。 据此归纳出三个核心痛点 —— 查不到（只显示「已预约」，不显示此刻有没有人）、 找不到（知道在五楼仍要逐个比对）、管不住（占座缺客观依据，管理员只能巡场）。 同时明确服务边界：只做感知与引导，不改门禁硬件、不涉及支付、不替代排课。**阶段调整**：最初设想用摄像头做占用识别，评估后放弃 —— 遮挡、隐私、每点位一路流的成本都不可控，改为低成本传感器方案。

② 概要设计。 确定四层架构：感知层（ESP32 + 传感器）、判定层（Flask 状态机）、 数据层（MySQL / SQLite + Redis）、应用层（Web / 终端 / 管理端）。 同时立下四条边界：硬件不做本地判定、状态只在服务端维护、变化实时推送、 AI 不参与判占。**阶段调整**：边界「状态只在服务端」是后来补上的 —— 早期把判定写进固件，换传感器型号就要改两处代码，规则开始漂移。

③ 详细设计。 逐模块定接口与数据结构：座位四态状态机（free / occupied / locked / error）的进出条件、 m/n/t 防占座参数（可锁定门槛 / 检测周期 / 有效回归时长）、 路网节点与边的存储格式、A* 的启发函数选型、推荐排序的四维权重。 **阶段调整**：传感器选型在真机阶段从 PIR 改为超声波 —— PIR 只能检测移动，人坐着不动会被判为空闲，这是原理性缺陷，只能换方案。

④ 编码实现。 后端采用 Flask + SQLAlchemy 组织业务，硬件用 Arduino 框架保证生态成熟； 前端不加构建链，Vue 3 直接引入，降低部署复杂度。 **阶段调整**：前端依赖原计划走公共 CDN，实测不可达导致整页空白，改为把 Vue / axios / Socket.IO / Font Awesome 全部自托管进仓库。

⑤ 测试。 后端以 pytest 建立自动化回归，覆盖判占规则、状态机转移、锁定流程、 A* 寻路与断点搭桥、路网生成、接口契约等，目前 **476 项全部通过**。 测试遵循一条原则：*修掉一个缺陷，就把复现它的用例固化成测试* —— 例如「断点相距 9999 像素不应被桥接」「模拟器关闭后不应继续写库」都是这样沉淀下来的。 硬件侧以真机联调为主：单台原型机跑通注册、绑定、拉配置、上报、 状态变化、后台可见的完整链路，并用串口诊断区分密码错误、信号问题与协议不匹配。

⑥ 部署。 云端采用 gunicorn + nginx，数据库 MySQL，静态资源由 nginx 直发； 设备侧支持产线批量烧录（把 WiFi 与服务器地址预置进固

件，烧完通电即连），现场只需安装与上电。**阶段调整**：云端启用 HTTPS 后，nginx 对 http 全站 301 跳转，而设备端 HTTP 客户端不带 TLS 也不会跟随跳转，导致设备全部失联 —— 固件改为按 URL 前缀自动选择 TLS 或明文后恢复。

六个阶段中有三次因为实测结果推翻原设计（摄像头改传感器、PIR 改超声波、CDN 改自托管）。这三次调整都记在第四章「难点」里，因为它们比顺利实现的部分更能说明问题。

四、作品实现、难点及特色分析

（1）作品实现及难点

1）作品实现

① 感知层实现。基于 ESP32 + Arduino 框架编写固件（PlatformIO 构建，主程序 467 行）。节点启动后进入配置流程：连接 2.4GHz Wi-Fi → 向云端注册设备 → 周期性拉取运行配置（绑定座位标签、超声判定阈值、上报间隔）→ 循环测距并按阈值判定座位是否有人 → 通过 HTTP JSON 上报。为避免 Wi-Fi 断线导致长时间失联，固件增加了服务器不可达自动重连与配置自动恢复逻辑。

② 云端实现。后端为 Flask 应用（app.py 约 4,556 行），按「模型层—工具层—接口层」组织：models 定义学校、建筑、楼层、座位、预约、传感设备等数据模型；utils 封装导航（路网加载、A* 寻路、跨层拼接）、推荐引擎、行为跟踪、AI 服务、图像预处理等能力；接口层对外提供 REST API 与 WebSocket 事件。系统采用「运行时配置」设计，管理员在后台修改的参数（AI 服务商与密钥、传感阈值、终端显示位置、离线判定时长等）即时生效，无需重启进程。

③ 移动端实现。前端为 Vue 3 单页应用（原生 JS，无构建步骤），通过 Capacitor 7 封装为 Android 应用。原生能力统一封装在 `native-bridge.js` 中，向页面暴露 `window.Native` 对象（扫码、语音识别、本地通知、地理定位、本地存储、震动），并提供「浏览器环境自动降级」策略 —— 同一套页面在浏览器里打开不会报错，只是原生功能不可用。移动端外壳（顶部细条导航、底部五个 Tab、侧边抽屉、悬浮按钮长按拖动）全部限定在 `body.app-mode` 作用域内，网页版界面不受影响。

④ 测试与质量保障。建立了覆盖接口、权限、导航、传感、前端静态体检的自动化测试套件（共 447 项，全部通过，测试代码 5,500 余行）。其中「前端静态体检」用于拦截一类难以排查的问题：模板中绑定的方法在脚本

里并不存在 —— 这类错误会被事件回调静默吞掉，表现为「按钮点了没反应」，该测试在提交前即可发现。

⑤ 平面图自动生成（降低部署门槛）。室内导航依赖平面图与路网，而普通教室、自习室往往没有现成的 CAD 图纸。系统为此提供「自动建图」：上传一段俯视或缓慢平移拍摄的室内视频、或多张照片，由 OpenCV 完成**关键帧提取 → 全景拼接 → 墙体线段识别 → 合并共线墙与过滤误检 → 生成可直接配置座位的楼层平面图**。同时保留「手动上传 CAD 图」通道，并给出拍摄建议（画面有纹理、帧间重叠 ⩾50%），把「没有图纸就没法用」这个落地障碍降到最低。

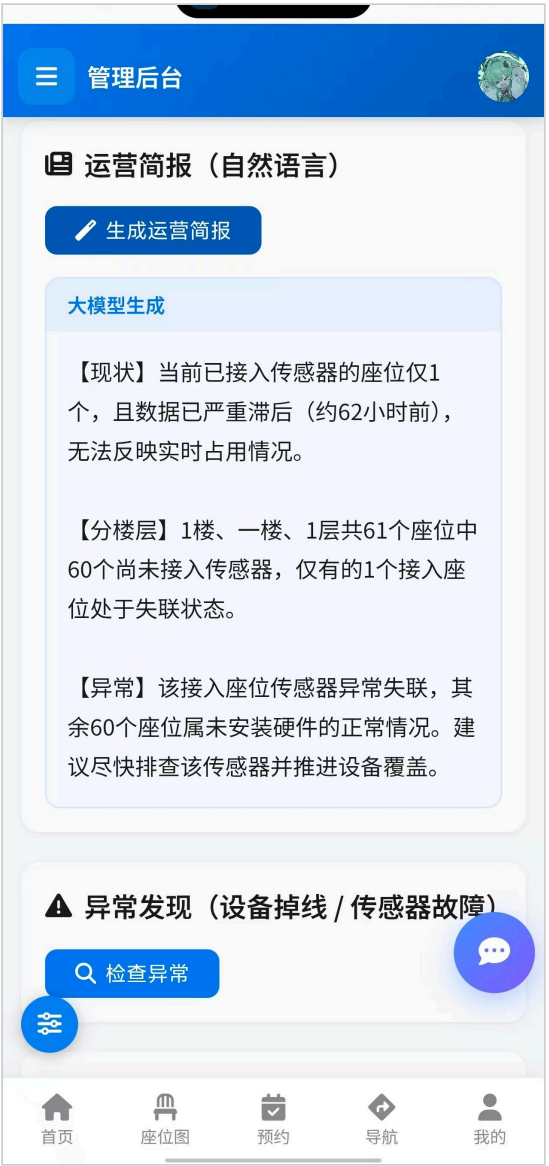


图 5　管理后台 · AI 运营简报（大模型基于实时传感数据生成）

图 5 说明：这是管理端「AI 运营简报」的实际输出。可以看到三点：① 简报里的数字（61 个座位、62 小时前）全部来自数据库真实状态，不是模板文案；② 大模型明确区分了「传感器异常失联」与「未安装硬件的正常情况」，

并把 60 个未接入传感器的座位归到后者 —— 这正是本作品的 AI 诚实性设计：宁可如实说「没有数据」，也不把缺失当成故障；③ 结论直接落到行动建议（「建议尽快排查该传感器并推进设备覆盖」）。管理端另提供「异常发现」快捷入口，可一键检查掉线设备与传感器故障。

2）难点及突破

难点一：室内定位的成本与精度矛盾。

室内没有 GPS 信号，常规做法是布设蓝牙信标或 UWB 基站，代价是需要逐点安装、供电、标定与长期维护，在学校场景难以规模化。**突破：**本项目提出「扫码锚定 + 短距惯性推算」方案 —— 座位上已有的二维码本身就是精确已知坐标的锚点，用户扫码即完成一次绝对定位；两次扫码之间，用手机自带的加速度计做步态峰值检测推算步数与距离、用陀螺仪／磁力计估计朝向，形成连续位置。该方案不增加任何硬件成本，误差随时间累积的问题通过「每走一段扫一次码重新锚定」化解。为降低使用门槛，面板默认收起为一个小胶囊，需要时展开。

难点二：超声波模块的供电与接线约束。

HC-SR04 的常规用法是 5V 供电，而 ESP32 的 GPIO 为 3.3V 电平，直接连接 ECHO 存在损坏风险，因此教科书方案是加装分压电阻。但分压电路增加了接线复杂度与故障点，在几十上百个节点规模下会显著抬高部署与维护成本。**突破：**团队通过回环自检（短接 TRIG 与 ECHO，捕获到 14μs 脉冲，证明 GPIO 输出与中断捕获通路正常）与实机对比测试，确认采用 **VCC→3V3、TRIG→GPIO16、ECHO→GPIO27 直连、免分压** 的接法可以稳定出数：实测坐下时距离约 3.59cm（判有人），空置时约 283cm（判无人）。3.3V 供电的代价是量程缩短，但检测 50cm 以内的座位占用完全够用 —— 这是一次「以场景约束换取部署简化」的有效工程权衡。

难点三：信息断在「最后一米」。

多数同类系统只解决「查询」，用户看到「3 楼 A 区有空位」之后仍需自行摸索。**突破：**把感知、决策与导航串成闭环 —— 座位状态由传感节点实时驱动，AI 负责把状态转译成建议，用户确认后直接由路径规划给出可走路线，并在移动端持续提供位置与朝向参考。整条链路在同一个 App 内闭合，不需要用户在多个系统之间搬运信息。

难点四：真实环境的不确定性。

现实部署中经常出现「传感器没装齐、地图密钥没配、网络时断时续、页面不在 App 内打开」等情况，若系统在这些条件下直接报错，演示和实际使

用都会中断。**突破：**全链路引入分级降级策略（详见创新点四），保证任何一种缺失都只影响对应能力，其余功能照常可用。

（2）特色分析

核心创新集中在「感知—决策—导航」闭环与零信标室内定位两处，均有可复现的实现与实测数据支撑。

创新点一：零信标室内定位 —— 用「已有的二维码 + 手机自带传感器」替代蓝牙信标

本系统提出并实现了**不依赖任何新增硬件**的室内定位方案：以座位上已有的座位二维码作为绝对坐标锚点，以手机内置的加速度计／陀螺仪／磁力计作为相对位移来源，二者结合输出连续位置。

- **锚定层：**用户扫描座位上的二维码即可获得该座位的精确平面坐标，完成一次绝对定位；系统同时把该节点写入本机，供签到校验等场景复用。
- **推算层：**加速度模长超过阈值且距上一步超过最小间隔时记一步，按平均步长（0.65m）累加距离；朝向由磁力计／陀螺仪给出，面板实时显示步数、推算距离与朝向。
- **误差控制：**惯性推算存在累积误差，方案不回避这一点，而是通过「每走一段重新扫码锚定」把误差控制在可用范围内，并在界面明确提示校准方式。

创新性所在：同类系统的室内定位几乎都以「增加硬件」为前提（蓝牙信标、UWB、Wi-Fi 指纹），本方案把定位成本降为零，使室内导航得以在普通教室、自习室等预算有限的场景规模化落地。换来的主要是**成本结构上的变化**，精度并不比蓝牙信标方案更高。对校园这类「面积大、预算有限、点位多」的场景，成本结构往往才是能否落地的决定因素。

创新点二：超声波模块 3.3V 单电源直连方案（免分压）

团队通过回环自检与实机对比，验证了 HC-SR04 采用 **VCC→3V3、TRIG/ECHO 直接连接 ESP32 GPIO（不接分压电阻）** 的可行性，实测数据：回环自检捕获到 14μs 脉冲（证明 GPIO 输出与中断捕获通路正常）；实机测距中，有人时约 3.59cm、无人时约 283cm，判定阈值 50cm 下状态切换稳定。

创新性所在：常规方案为规避 5V 电平损坏 3.3V GPIO 的风险，普遍加装分压电阻。本方案在明确边界（3.3V 供电量程缩短，但座位占用检测只需 50cm 内）的前提下主动简化硬件，**减少两个元件、两个焊点与一类故障源**，使单节点更易复制、更少维护。同时，这种「先证明通路正常、再判定传感器本体故障」的回环自检方法，本身也可作为硬件调试的标准步骤复用到其他传感项目。

创新点三：把「AI 诚实性」作为设计原则，而非把缺失数据说成故障

系统接入大语言模型后，团队发现一个容易被忽视的问题：当大部分座位尚未接入传感器时，模型会把「没有数据」解释成「传感器异常」，输出诸如「其余 13 个座位传感器异常」的错误结论，误导使用者。

为此，系统在数据层与提示层同时做了约束：① 离线判定只作用于「已绑定传感设备」的座位，未接入传感器的座位不参与离线统计；② 推送给模型的快照中显式区分「未接入传感器」与「传感器故障」两类状态，并在系统提示词中明确要求不得混淆；③ 前端对无传感器座位单独标识，不用「异常」措辞。

创新性所在：多数作品的 AI 止步于「接上模型、能问答」。本作品把**数据的可解释边界**当作一等设计对象 —— 让 AI 在数据不足时如实说「不知道」，而不是编造一个看起来合理的结论。对于要把 AI 结论用于实际决策的系统，这种「诚实性」比语言流畅度更重要。

创新点四：全链路分级降级 —— 任何一项缺失都不导致整体失效

系统在五个层面都建立了「有则用、无则退」的降级链，且降级过程对使用者透明：

层面	降级链
室内路网	已保存路网文件 → 按楼层座位坐标自动生成 → 按平面图尺寸铺兜底网格。三级都只作用于内存，不覆盖管理员配置的正式路网。
室内定位	扫码锚定 → 惯性推算 → 手动选点 / 点击地图吸附最近路网节点。
室外导航	高德地图步行路线 → 无密钥时自绘相对方位图（自己在圆心、建筑按真实方位与距离摆放，完全离线可用）→ 一键跳转手机地图 App。
移动端能力	App 内调用原生扫码／语音／通知／定位 → 浏览器中自动降级为安全返回，页面不报错、功能不中断。
前端依赖	Vue／axios／Socket.IO／图标字体全部自托管在 `static/vendor/`，**不依赖任何外部 CDN**。

关于「前端依赖」这一层，来自一次真实的踩坑：系统最初把 Vue、axios、Socket.IO 和图标字体都指向公共 CDN（cdn.jsdelivr.net）。调试期间发现页面**时而正常、时而满屏未渲染的模板占位符**。排查后确认根因是：CDN 一旦不可达，Vue 加载失败，页面脚本在执行 `createApp()` 时抛出异常，整个页面的响应式渲染就此中断 —— 而浏览器不会给出任何显式提示，使用者只会看到一堆类似 `${ xxx }` 的原始文本。经多次实测，该 CDN 在测试网络下**连续 6 次请求全部失败**。

如果依赖的第三方 CDN 在现场不可用，作品将无法正常展示。为此我们把全部前端依赖下载到项目内自托管，并在测试套件中加入了「任何模板不得再出现外部 CDN 引用」的回归断言，从机制上杜绝此类问题再次出现。

创新性所在：降级做成了贯穿五层的统一设计，没有分散成各处零散的容错代码。它带来的直接价值是**演示与部署的鲁棒性**：现场网络异常、地图密钥缺失、传感器尚未装齐、第三方 CDN 不可用，系统依然可以完整演示核心链路。这在实际竞赛与真实部署中都是高频场景，而「自托管依赖、不把演示成败交给第三方」本身也是工程成熟度的体现。

创新点五：感知—决策—导航一体化闭环

本作品的链路从传感节点一直贯到到座核验：**传感节点驱动座位状态 → 云端调度与鉴权 → AI 转译为可执行建议 → 用户一键确认 → 路径规划与移动端导航 → 到座扫码核验**。每一环的产出都是下一环的输入，用户从「想找个座位」到「坐下并完成签到」不需要离开同一个 App，也不需要自行搬运信息。

创新性所在：针对行业普遍存在的「信息断在最后一米」问题给出的系统性解法 —— 把感知层的实时性、决策层的智能性与移动层的便携性真正连接起来，而非各自独立。

（3）核心代码与解析

下面按「算法核心 → 定位核心 → AI 核心 → 硬件数据核心」的顺序，摘录本作品的四段关键实现并逐段解释。代码均为本团队自主编写。

3.1 室内路径规划：A* 寻路与断点自动搭桥

路径规划建立在「节点 + 边」的路网上（`nodes{id,x,y,type}` 与 `edges[{from,to}]`），由管理员在平面图上手绘生成。启发函数取两节点间的欧氏距离，对平面图这种二维场景是可采纳（admissible）的，因此 A* 能找到最短路径。

```
def find_path(self, start_node, end_node):
    open_set  = [(0, start_node)]                  # 待考察节点堆，按估算总代价排序
    came_from = {}                                 # 前驱表：记录每个节点是从哪走来的
    g_score   = {start_node: 0.0}                  # 起点到该节点的实际已走代价
    f_score   = {start_node: self.heuristic(start_node, end_node)}
```

```
    while open_set:
        _, current = heapq.heappop(open_set)        # 取估算总代价最小的节点
        if current == end_node:
            path = []                               # 从终点沿前驱表回溯出完整路径
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start_node)
            path.reverse()
            return path, g_score[end_node]

        for neighbor in self.adjacency.get(current, []):
            candidate = g_score[current] + self.heuristic(current, neighbor)
            if neighbor not in g_score or candidate < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor]   = candidate
                f_score[neighbor]   = candidate + self.heuristic(neighbor, end_node)
                heapq.heappush(open_set, (f_score[neighbor], neighbor))

    return [], 0.0                                  # 无路径
```

解析： `g_score` 记录「已经走掉的真实距离」，`f_score = g + 启发值` 是对「经过这个节点到终点还要走多远」的估计。每轮取出 `f` 最小的节点展开，一旦展开到终点，路径就是最优的。和前驱表 `came_from` 配合回溯，就得到完整的节点序列。

断点自动搭桥： 实测中遇到过管理员少画一条边、路网断成两块的情况 —— 表现为「起点和终点之间没有连通路径」，用户完全不知道哪里出了问题。处理办法是先做连通分量分析，若多于一块，就用一条最短的边把最近的两个缺口连起来：

```
def _bridge_components(self):
    comps = self.components()                       # 先做连通分量分析
    if len(comps) <= 1:
        return []
    bridges = []
    # 每次取「分属不同分量且距离最近」的一对节点连起来，直到全部连通
    while len(self.components()) > 1:
        best = None
        for a in 分属分量A的所有节点:
            for b in 分属分量B的所有节点:
                d = self.heuristic(a, b)
                if best is None or d < best[0]:
                    best = (d, a, b)
        self.adjacency[best[1]].append(best[2])     # 只改内存邻接表
        self.adjacency[best[2]].append(best[1])     # ★ 只改内存，不写回磁盘
        bridges.append({'from': best[1], 'to': best[2]})
    return bridges
```

解析： 搭桥**只作用于内存中的邻接表，不会写回路网文件** —— 否则管理员的原始数据会被程序悄悄改掉。同时把搭桥情况作为 `network_note` 回传给前端

如实展示：「路网存在 1 处断点，已自动连接（p25→p28）后规划成功。建议在『平面图与路网配置』里把路线画通，以免绕行。」**能救急，但不替用户掩盖问题。**

3.2 零信标室内定位：自适应阈值的步态检测

室内定位用手机自带的加速度计做步态检测、再用磁力计／陀螺仪估朝向，只在「检出一步」时积分位置。整个方案里最难定的是**「多大算一步」这个阈值**，积分公式本身并不复杂。

```
onMotion(e) {
  const a = e.accelerationIncludingGravity || e.acceleration;
  const mag = Math.sqrt(a.x*a.x + a.y*a.y + a.z*a.z);   // 加速度模长
  const now = Date.now();

  // ★ 泄漏式最小值包络：跟着「静止水平」走，不被走路峰值带走
  //   · 被更小的值拉低（捕捉静止水平）
  //   · 自身缓慢回升（每帧 +0.02，约每秒 1.2）
  this.floor = Math.min(mag, this.floor + 0.02);
  const thr  = Math.min(14.0, Math.max(9.5, this.floor + 1.7));

  // 简易峰值检测：越过阈值、且距上一步足够久 => 记一步
  const over   = mag > thr && this.lastMag <= thr;      // 越过阈值（上升沿）
  const enough = (now - this.lastStepAt) > this.STEP_MIN_MS;  // 距上一步够久
  if (over && enough) {
    this.steps++;
    this.distM += this.STEP_LEN;                        // 平均步长 0.65 m
    this.lastStepAt = now;
    this.recomputeFromSteps();                          // 按朝向积分出位置
    this.render(); this.emit();
  }
  this.lastMag = mag;
}
```

解析 —— 这个阈值被真机实测改了三版：

- **第一版：写死 13.5。**实测发现本机静止时加速度模长只有 8.6（低于理论重力 9.8），正常走路的峰值也就 10¯11.5 —— **一步都记不上**。写死一个魔数在换手机、换握法后就失效。
- **第二版：低通滤波。**改成 `base = base*0.996 + mag*0.004`，让基线自适应。实机走十几步只记到 8 步 —— 因为**走路的峰值会把基线整体抬高**，阈值跟着涨，越走越不记步。低通滤波求的是「均值」，而我们需要的是「地面」。
- **第三版（现用）：泄漏式最小值包络。**取历史最小值、并让它以极慢速度回升。走路时每次落脚都会把它压回静止水平，因此它始终贴着地面走，不会被峰值带走。换成这台手机实测，阈值自动落在 10.4 左右，步数正常记录。

位置积分：只在「检出一步」时才调用 `recomputeFromSteps()`，用当前朝向把步数换算成位移：

```
recomputeFromSteps() {
  const rad = this.heading * Math.PI / 180;
  const d   = this.distM * this.pxPerM;          // 米 → 平面图像素
  this.posX = this.anchorX + d * Math.sin(rad);  // 锚点来自扫码，是绝对坐标
  this.posY = this.anchorY - d * Math.cos(rad);  // 屏幕 y 轴向下，故取负
}
```

解析：朝向事件每秒触发二十来次且抖动很大，若拿它直接积分位移，人会原地乱飘 —— 那不是惯性推算，是噪声。**只在步态事件上积分**，等于用「一步」这个物理事件做了一次天然低通滤波。另外 `anchorX/anchorY` 来自扫码得到的座位绝对坐标，误差因此**不随时间累积**，走一段扫一次即可重置。

3.3 AI 智能体：工具调用循环与写操作二次确认

本作品的 AI 能查真实数据、也能修改系统状态，不止于一个聊天框。它通过 OpenAI 风格的 function calling 调用 8 个工具（5 个只读、3 个写操作）。**关键设计是：写操作在智能体循环里永不执行。**

```
MAX_ROUNDS = 4        # 工具调用最多几轮，防止模型来回兜圈子

for rnd in range(1, MAX_ROUNDS + 1):
    res   = llm.chat_with_tools(messages, tools=all_tools)
    calls = res.get('tool_calls') or []
    if not calls:
        break                                  # 模型不再调工具 = 可以作答了

    for call in calls:
        name = call['function']['name']
        args = json.loads(call['function']['arguments'] or '{}')

        if name in WRITE_TOOL_NAMES:
            # ★ 写操作：只记成「提议」，绝不在这里执行
            if not ctx.get('is_admin'):
                ...  # 权限不足，直接回一条错误给模型
                continue
            proposals.append({'tool': name, 'args': args,
                              'desc': describe_action(name, args)})
            messages.append({'role': 'tool', 'tool_call_id': call['id'],
                'content': json.dumps({
                    'status': 'pending_user_confirmation',
                    'note': '操作已记录，等待用户在界面上确认后才会真正执行。'
                            '请在回答里说明你准备做什么，不要声称已经完成。'
                }, ensure_ascii=False)})
            continue

        # 只读工具：直接执行，把结果回喂给模型
        messages.append({'role': 'tool', 'tool_call_id': call['id'],
                         'content': json.dumps(run_read_tool(name, args, ctx),
                                               ensure_ascii=False)})
```

解析：大模型会理解错意图，这是概率问题、不是 bug。如果让它直接改数据，一次误判就可能关掉一整层的座位。这里的做法是：

- **只读工具直接执行**（查座位、查设备在线状态），有结果才能继续推理；
- **写工具只落进 `proposals` 列表**，随回答一起返回前端，渲染成一张确认卡片；用户点了「确认执行」才调 `/api/ai/agent/confirm` 真正落库。
- 工具返回值里显式告诉模型 `pending_user_confirmation` 并叮嘱它「不要声称已经完成」—— 否则模型会在回答里说「已为您关闭 12 个座位」，而实际上什么都没发生，用户就被骗了。
- 写操作还有一道管理员权限校验，普通用户提问时直接返回「只有管理员可以执行这个操作」。

这样即使模型完全理解错了，最坏结果也只是**弹出一张错误的提议卡片**，用户点「取消」即可，数据毫发无损。

3.4 感知层数据：区分「未接入传感器」与「传感器故障」

这是本作品「AI 诚实性」设计原则的落地点。系统里并非每个座位都装了传感器 —— 未接入的座位如果参与「静默」统计，就会被 AI 报告成「传感器故障」，于是管理端会看到一条吓人的假警报。

```
no_sensor_count = 0
instrumented = set()          # 已接入传感器设备的座位集合

for seat in seats:
    if seat.id not in instrumented:
        # ★ 未接入传感器的座位：单独计数，不参与「静默/异常」判定
        no_sensor_count += 1
        continue
    if seat.last_seen 超过离线阈值:
        offline_instrumented.append(seat)         # 这才是真正的异常

snapshot = {
    'total':            total,
    'no_sensor_count':  no_sensor_count,          # 未安装硬件（正常）
    'instrumented':     total - no_sensor_count,  # 真正在监测的
    'offline':          offline_instrumented,     # 传感器失联（要排查）
    ...
}
```

解析：同一个「没有数据」，背后可能是两种完全不同的现实 ——「这里根本没装设备」（正常）和「装了但掉线了」（故障）。如果不加区分，AI 会把 60 个未装硬件的座位报成 60 个故障，管理端被假警报淹没，真故障反而被埋掉。

因此在喂给模型的快照里就**把这两类显式分开**，并在提示词里要求模型不得混淆。实际输出（见「图 5　管理后台 · AI 运营简报」）中，模型正确地区

分了两者:

> **【异常】**该接入座位传感器异常失联，**其余 60 个座位属未安装硬件的正常情况**。建议尽快排查该传感器并推进设备覆盖。

解析：这句话是整个设计原则最好的验证 —— **模型没有被引诱去编一个「60 个座位传感器故障」的结论**，而是如实说明了数据边界。这比让 AI 说漂亮话更有工程价值。

五、团队介绍和人员分工

所在学校	沧州交通学院
团队名称	苗苗
团队人数	3 人

角色	姓名	主要分工
队长	吴嘉森	总体方案与进度统筹：制定开发计划与里程碑，组织技术研讨与复盘，对接指导教师与赛务，负责答辩材料与现场演示。技术侧承担云端开发与部署运维 —— 数据模型设计、REST API 与 WebSocket 事件、座位四态状态机与判占规则、预约鉴权与多校权限体系、AI 服务层与智能体工具层（工具定义、写操作确认机制）、云端部署与设备接入（HTTPS 接入、产线预置配置方案）。
队员	刘彭宇	硬件感知与算法实现：ESP32 固件开发与调试、传感器选型与接线验证（含 PIR 改为超声波的对照测试与接法确认）、设备注册与本地上报链路、失败自愈与配网流程；室内路网生成（平面图细化与节点采样）、A* 寻路算法实现、断点搭桥的距离上限设计与跨楼层路径串接、路径正确性与定位精度验证。
队员	李冠桦	前端与移动端开发：实时座位图、预约与签到、室内导航与路径渲染、室外导航、移动端外壳与原生能力封装（扫码／语音／通知／定位／传感器导航／长按拖动）；同时负责测试与文档 —— 自动化测试套件编写与维护、功能与兼容性测试、缺陷跟踪与回归验证、设计文档与演示材料制作。
指导教师	范雯	选题方向与技术路线指导、阶段性评审与改进建议、资源协调。

分工说明：三人团队按「云端—硬件与算法—前端与移动端」三层切分：队长统筹全局并负责云端判定与权限，刘彭宇负责感知层与路径算法，李冠桦负责全部界面与端侧封装。三人在接口约定后并行开发，跨层问题（如判占阈值与定位精度、路网节点与前端渲染）共同研讨定案。项目全流程由三人共同参与方案讨论与验收测试，代码统一提交至同一仓库，通过自动化测试与真机联调共同把关。

六、其他

（1）运行环境与版本号

类别	具体平台 / 语言 / 工具及版本号
服务端语言与运行时	Python 3.14.2
服务端框架	Flask 3.1.0; Flask-SQLAlchemy 3.1.1; Flask-SocketIO 5.5.1; Flask-CORS 5.0.1
数据库与缓存	MySQL 8.0（PyMySQL 1.1.1）; Redis 5.2.1
WSGI 服务器与反向代理	gunicorn 23.0.0（eventlet worker）; nginx（HTTPS 反向代理）
图像与算法库	OpenCV 4.11.0.86; NumPy 2.2.4; Pillow 11.1.0; qrcode 7.4.2
前端	Vue 3.4.21（CDN 引入，无构建步骤）; axios 1.6.7; Socket.IO Client 4.7.5; 原生 JavaScript + CSS
移动端封装	Capacitor Core / Android / CLI 7.x（本项目实测 7.6.9）
移动端插件	@capacitor-mlkit/barcode-scanning 7.x（扫码）; @capacitor-community/speech-recognition 7.x（语音）; @capacitor/geolocation 7.x; @capacitor/local-notifications 7.x; @capacitor/preferences 7.x; @capacitor/app 7.x
Android 构建	compileSdk 35 / targetSdk 35 / minSdk 23; JDK 21（Temurin）; Gradle 8.11.1
硬件平台	ESP32（esp32dev，ESP32-D0WD 双核 240MHz）; HC-SR04 超声波测距模块; Arduino 框架; PlatformIO 构建; ArduinoJson 7.x
硬件接线	HC-SR04 VCC→ESP32 3V3; TRIG→GPIO16; ECHO→GPIO27（直连，免分压）

AI 大模型	DeepSeek（服务商 deepseek；模型 **deepseek-chat**；接口 https://api.deepseek.com/v1）
地图服务	高德地图 JavaScript API 2.0（含无密钥降级方案）。地图数据来源：高德地图（AutoNavi），审图号 **GS(2025)5996号**，随底图左下角原样展示
公网访问地址	https://zhinengzuo.site

（2）核心技术说明

1）占用判定算法

座位状态由「最近一次上报」与「上报新鲜度」共同决定：① 若座位最近一次上报为「有人」，状态判定为已占用；② 若为「无人」且距上次上报时间在离线阈值内，判定为空闲；③ 若上报时间超过阈值，则视为设备掉线，标记为异常。为避免「长时间无上报后又收到一条空报告」把已占用座位错误回退，离线判定仅在报告为「无人」时生效。超声波模式下，距离小于阈值（默认50cm）判定为有人。

2）室内路径规划

每个楼层对应一份路网数据（节点坐标 + 邻接边）。定位时把用户点击／扫码位置吸附到最近路网节点，再以 A* 启发式搜索（启发函数为节点间欧氏距离）求解最短路径；跨楼层时先分别求各层路径，再通过楼梯口／电梯口节点拼接，并给出「上楼／下楼」文字提示。当楼层没有路网文件时，系统按座位坐标或平面图尺寸即时生成临时路网，仅驻留内存，不覆盖管理员配置。

3）实时推送

座位状态变化通过 Flask-SocketIO 广播 `seat_update` 事件，前端订阅后局部更新对应座位，避免整页轮询。智能终端大屏采用「座位状态 2.5 秒 / AI 文案 30 秒」的双频率刷新，并用 `seats_only` 参数跳过 AI 调用，保证大屏在大模型响应较慢时仍能流畅刷新。

（3）AI 使用情况

本作品开发过程中使用了 AI 辅助，情况如下。

1）作品中集成的 AI 能力（属于作品功能）

项目	说明
AI 大模型名称及版本	DeepSeek，模型 **deepseek-chat**

调用方式	通过 HTTPS 调用 DeepSeek 开放接口（api.deepseek.com/v1），服务端封装为可切换适配器
集成位置	用户端 AI 助手浮窗、首页与终端的「一句话结论」、座位推荐理由、管理端 AI 运营中心
是否可替换	可。管理后台可现场切换服务商与模型，支持接入本地部署的模型服务
数据边界	发送给模型的仅为座位状态统计与结构化快照，不含用户姓名、学号等个人身份信息

2）开发过程中使用 AI 辅助编程的情况

项目	说明
是否使用 AI 辅助编程	是
所用 AI 模型及版本	DeepSeek（deepseek-chat 系列模型），用于代码补全、错误排查建议与文档草拟
AI 生成代码占比（估算）	**约 50%**（按代码行数估算，含 AI 补全与改写的部分；所有 AI 参与的代码均经团队成员逐行阅读、修改与实测验证后方才合入）
由团队自主设计与实现的部分	系统总体架构与三层划分；数据模型设计；占用判定规则与离线释放策略；室内路网结构与 A* 寻路及跨层拼接方案；零信标室内定位方案（扫码锚定 + 步态推算）的设计与参数标定；ESP32 固件流程与超声波 3.3V 直连方案的验证；多校权限模型；AI 提示词与「数据边界诚实性」约束规则；全部界面设计与交互流程；测试用例设计
AI 参与的部分	部分重复性代码的补全与改写、接口样板代码、样式微调、报错信息的排查线索检索、文档措辞润色
团队复核情况	所有 AI 参与的代码均经团队成员逐行阅读、修改与实测验证后方才合入；核心算法与业务规则由团队自行编写

（4）第三方开源代码

本作品使用以下第三方开源组件，均在各自许可证范围内使用，特此说明：

组件	版本	用途与许可证
Vue.js	3.4.21	前端响应式框架，MIT License
axios	1.6.7	HTTP 请求库，MIT License
Socket.IO Client	4.7.5	实时通信客户端，MIT License
Font Awesome Free	6.5.1	图标字体，CC BY 4.0 / SIL OFL 1.1 / MIT

组件	版本	用途与许可证
Flask / Flask-SQLAlchemy / Flask-SocketIO	3.1.0 / 3.1.1 / 5.5.1	Web 框架与扩展，BSD-3-Clause
SQLAlchemy	2.x	ORM，MIT License
PyMySQL	1.1.1	MySQL 驱动，MIT License
gunicorn	23.0.0	WSGI 服务器，MIT License
OpenCV	4.11.0.86	平面图图像处理（路网提取），Apache-2.0
NumPy	2.2.4	数值计算，BSD-3-Clause
Pillow	11.1.0	图像读写，HPND License
qrcode	7.4.2	座位签到二维码生成，BSD License
Capacitor	7.x	Web 应用封装为原生 App，MIT License
Capacitor 官方与社区插件	7.x	扫码（MLKit，Apache-2.0）、语音识别、地理定位、本地通知、本地存储、App 信息，均为 MIT / Apache-2.0
ArduinoJson	7.x	ESP32 端 JSON 构造与解析，MIT License
Arduino-ESP32 框架	Espressif32	ESP32 开发框架，LGPL-2.1 / Apache-2.0
高德地图 JavaScript API	2.0	室外地图与步行路线（第三方服务，非开源，按其开放平台条款使用）

除上述组件外，本作品的系统架构、业务逻辑、算法实现、界面设计与硬件方案均为团队原创。第三方组件均以库／框架形式调用，未修改其源码。

（5）操作步骤与测试账号

测试账号

超级管理员：**用户名 admin　密码 SuperAdmin@123**（用于查看全部管理功能）
普通用户：可在登录页点击「注册」自行注册（选择「普通用户」身份，注册后即可直接登录）

访问方式一（推荐，无需安装）： 浏览器打开 https://zhinengzuo.site ，使用上述账号登录。

访问方式二（Android App）： 安装随作品提交的 APK 文件（智座.apk），打开即可使用，功能与网页版一致并额外支持扫码、语音、通知等原生能力。

核心功能演示步骤:

1. **查看实时座位**: 登录后首页即显示当前场馆的座位状态摘要与 AI 建议; 点击底部「座位图」查看平面图上的分色座位。
2. **找空座（一句话）**: 在 App 内点右下角 AI 助手图标旁的语音按钮，说「找个空座」，系统会挑出空闲座位并自动跳转导航页。
3. **室内导航**: 进入「导航」页，起点选择「手动选点」并点击地图任意位置（或选择「入口」自动定位到入口），终点选择一个座位，点击「规划路径」，地图上会绘制蓝色路径并标注「我的位置」与「终点」。
4. **传感器导航（PDR）**: 在导航页右侧点击收起的「传感器导航」胶囊展开面板，走动可看到步数与推算距离变化; 点击「扫桌上的码锚定」可用座位码重新校准。
5. **扫码占座**: 在「座位图」页点击右下角「扫码占座」，对准座位上的二维码即可完成占座。
6. **预约与签到**: 进入「预约」页选择座位与时长提交预约; 在「我的预约」中可查看预约二维码并点击「签到」完成核验。
7. **室外导航**: 进入「室外导航」页点击「获取我的位置」，页面会显示你的位置与目标建筑的方位和距离; 已配置地图密钥时同时显示步行路线。
8. **智能终端大屏**: 浏览器打开 https://zhinengzuo.site/terminal ，可查看面向场馆入口的只读展示页，支持切换楼层。
9. **管理功能**: 用超管账号登录后，顶部导航进入「管理」，可依次体验场所与楼层管理、平面图与路网配置、传感设备监控与调试、学生批量导入、AI 运营中心与系统设置。

注: 演示环境的传感器节点数量有限，未接入传感器的座位会明确标注为「未接入传感器」，不影响其余功能的演示。

（6）设备批量部署与测试规模

产线批量部署。逐台开热点配网在一层楼几十个座位的规模上不可行， 因此固件支持把配置预置进固件: 管理后台填写 WiFi 名称、密码与服务器地址后生成 `secrets.h`，编进固件即可。所有设备烧同一份二进制，烧完通电自动连接并注册， 不需要人工逐台配置。预置值在编译期写入，NVS 为空时回落使用; 现场配网过的设备优先使用 NVS 中的配置，换 WiFi 时现场配一次即可，不必重烧。

设备自愈。设备连不上 WiFi 或服务器不可达达到阈值后， 会自动清空配置并重新开启配网热点等待人工配置，不会因为配置失效而变成无法使用的砖机。

在线状态判定。设备每 60 秒会注册一次并拉取配置，这些心跳不能等同于「数据在更新」。系统因此把「最后心跳时间」与「最后上报时间」分成两个字段独立记录，面板上区分三种状态：在线（数据在更新）、无数据（心跳正常但不上报，多为未绑定座位或传感器未接好）、离线（心跳也断了）。这样可以避免设备不上报数据却长期显示「在线」的误导。

测试规模。后端自动化测试 **476 项全部通过**，覆盖判占与状态机、锁定与有效回归、A* 与断点搭桥、路网生成、推荐排序、接口契约、前端产物校验等。硬件侧以单台原型机完成全链路真机联调；百台级的并发能力尚未做压力测试，这一点在第四章「难点与局限」中如实说明。

七、致谢

本项目的完成离不开许多人的帮助。

首先感谢我们的指导教师范雯老师。从选题方向的把握到技术路线的推敲，老师在每一个关键节点都给出了中肯的意见，尤其是在我们纠结于「要不要为了室内定位去布蓝牙信标」时，老师提醒我们回到真实场景去算成本账 —— 这句话直接促成了后来「零信标定位」这条技术路线的确立。

感谢学校为团队提供的场地与设备支持，让超声波传感器、ESP32 开发板这些硬件能够真正跑起来、测出数据，而不是停留在纸面方案上。

感谢图书馆的老师与同学们在测试阶段给予的配合。为了让传感器测出真实的坐下／离开数据，很多同学反复配合我们坐下、站起、走动，还帮我们指出了界面上一处措辞不当的问题（把「未接入传感器」写成「传感器异常」），这个提醒最终演变成了本作品一条重要的设计原则 —— 让 AI 在数据不足时如实说「不知道」。

感谢开源社区。本作品建立在 Vue、Flask、SQLAlchemy、OpenCV、Capacitor、Arduino-ESP32 等众多优秀开源项目之上，正是这些项目让一个学生团队能够把想法快速变成可运行的完整系统。

最后感谢大赛组委会提供这样一次把课堂知识落到真实问题上的机会。在反复调试传感器接线、排查导航定位偏差、为一个显示比例问题折腾到深夜的过程中，我们对「工程」二字的理解比任何一门课都要具体。